12.01 What is computational thinking?

Computational thinking is a problem-solving process where a number of steps are taken in order to reach a solution. It is a logical approach to analysing a problem, producing a solution that can be understood by humans and used by computers.

Computational thinking involves four key strands: abstraction, decomposition, data modelling, pattern recognition and algorithmic thinking.

Abstraction

Abstraction involves filtering out information that is not necessary to solve a problem. There are many examples in everyday life where abstraction is used. Figure 12.02 shows part of the underground map of London, UK. The purpose of this map is to help people plan their journey within London. The map does not show a geographical representation of the tracks of the underground train network nor does it show the streets above ground. It shows the stations and which train lines connect the stations. In other words, the information that is not necessary when planning how to get from one landmark to another is filtered out. The essential information we need to be able to plan our route is clearly represented.

Abstraction gives us the power to deal with complexity. An algorithm is an abstraction of a process that takes inputs, executes a sequence of steps, and produces outputs. An abstract data type defines an abstract set of values and operations for manipulating those values.

TASK 12.01

Use the aerial photograph in Figure 12.01 and draw a map just showing the essential details for finding a route from landmark A to landmark B.



Figure 12.01 Aerial photograph of part of a city

Decomposition

Decomposition means breaking problems down into sub-problems in order to explain a process more clearly. Decomposition leads us to the concept of program modules and using procedures and functions.

Data modelling

Data modelling involves analysing and organising data (see Chapter 13). We can set up abstract data types to model real-world concepts, such as queues or stacks. When a programming language does not have such data types built-in, we can define our own by building them from existing data types. There are more ways to build data models. In Chapter 27 we cover object-oriented programming where we build data models by defining classes. In Chapter 29 we model data using facts and rules. In Chapter 26 we cover random files.

Pattern recognition

Pattern recognition means looking for patterns or common solutions to common problems and using these to complete tasks in a more efficient and effective way. There are many standard algorithms to solve standard problems, such as sorting (see Section 13.03 and 23.03) or searching (see Section 13.02 and 23.04).

Algorithm design

Algorithm design involves developing step-by-step instructions to solve a problem.

KEY TERM

Algorithm: a sequence of defined steps that can be carried out to perform a task

12.02 What is an algorithm?

We use algorithms in everyday life. If you need to change a wheel on a car, you might need to follow instructions (the algorithm) from a manual.

1. Take a spanner and loosen the wheel nuts.
2. Position a jack in an appropriate place.
3. Raise the car.
4. Take off the wheel nuts and the wheel.
5. Lift replacement wheel into position.
6. Replace wheel nuts and tighten by hand.
7. Lower the car.
8. Fully tighten wheel nuts.

This might sound all very straightforward. However, if the instructions are not followed in the correct logical sequence, the process might become much more difficult or even impossible. For example, if you tried to do Step 1 after Step 3, the wheel may spin and you can't loosen the wheel nuts. You can't do Step 4 before Step 3.

If you want to bake a cake, you follow a recipe.

1. Measure the following ingredients: 200g sugar, 200g butter, 4 eggs, 200g flour, 2 teaspoons baking powder and 2 tablespoons of milk.
2. Mix the ingredients together in a large bowl, until the consistency of the mixture is smooth.